

NETWORK TRAFFIC ANALYSIS REPORT

Wireshark Network Traffic Capture and Analysis

Prepared By: Sathya V

Task: Task 8 – Capture Network Traffic with Wireshark

Tool: Wireshark 4.6.9

Platform: Windows

Network Interface: Wi-Fi

Status: Completed

Date: October 2026

1. Introduction

Network traffic analysis is an important part of network security. It helps security professionals understand how devices communicate, identify network protocols, troubleshoot connectivity problems, and detect suspicious activities.

For this practical task, **Wireshark** was used to capture and analyze live network traffic from a local Wi-Fi interface. Different display filters were applied to study TCP, DNS, HTTP, and encrypted TLS traffic.

The practical also focused on understanding the TCP connection establishment process and the security differences between HTTP and HTTPS.

2. Objective

The main objectives of this practical were:

- To install and configure Wireshark.
- To capture live network traffic.
- To analyze captured packets using Wireshark filters.
- To identify DNS traffic.
- To analyze TCP traffic.
- To understand the TCP 3-way handshake.
- To examine HTTP traffic.
- To understand why unencrypted HTTP is insecure.
- To understand how HTTPS protects communication.
- To export the captured traffic as a .pcap file.
- To perform network analysis in an ethical and authorized environment.

3. Tools and Technologies Used

Tool / Technology	Purpose
Wireshark 4.6.9	Packet capture and network analysis
Npcap	Network packet capture support
Windows	Operating system
Wi-Fi Interface	Live network traffic source
PCAP	Packet capture file format

4. Wireshark Installation

Wireshark was installed successfully on the Windows system. The installation included the required packet-capture support through Npcap.

After launching Wireshark, the available network interfaces were displayed. The Wi-Fi interface was selected for live packet capture.

Screenshot 1 – Wireshark Installation

Insert: 01_installation.png

Caption:

Figure 1: Wireshark 4.6.9 successfully installed with available network interfaces.

Observation:

The Wireshark application was successfully installed and the available network interfaces were detected.

5. Live Network Traffic Capture

The active Wi-Fi network interface was selected and live network traffic was captured for more than two minutes.

During the capture, multiple types of network communication were observed, including TCP, DNS, TLS, acknowledgements, and application data.

Screenshot 2 – Live Capture

Insert: 02_live_capture.png

Caption:

Figure 2: Live network traffic captured from the Wi-Fi interface using Wireshark.

Observation:

The packet list showed continuous network communication between different source and destination addresses.

6. HTTP Traffic Analysis

The following Wireshark display filter was applied:

http

The purpose of this filter was to identify plain HTTP packets in the captured traffic.

No matching HTTP packets were observed.

This is expected because most modern websites and applications use **HTTPS/TLS** rather than unencrypted HTTP.

Screenshot 3 – HTTP Filter

Insert: 03_http_filter.png

Caption:

Figure 3: HTTP display filter applied to the captured network traffic.

Result

The http filter did not return any matching packets.

Analysis

The absence of plain HTTP traffic indicates that the observed web communication was primarily encrypted using HTTPS/TLS.

No artificial or misleading HTTP traffic was generated for the purpose of producing evidence.

7. DNS Traffic Analysis

The following display filter was used:

`dns`

DNS stands for **Domain Name System**. It is responsible for resolving domain names into IP addresses.

For example:

`example.com` → IP Address

DNS communication may contain information such as:

- Requested domain name
- Query type
- Source address
- Destination DNS server
- Response information

Screenshot 4 – DNS Filter

Insert: 04_dns_filter.png

Caption:

Figure 4: DNS packets identified using the Wireshark dns display filter.

Observation

DNS traffic was observed during the capture. This demonstrates the role of DNS in resolving domain names before network communication with the destination server.

8. TCP Traffic Analysis

The following display filter was applied:

`tcp`

This filter displayed TCP packets from the captured traffic.

TCP is a connection-oriented transport protocol that provides reliable communication between network endpoints.

The capture contained several TCP packets including:

- ACK packets
- Application data
- TLS-related traffic
- TCP segments

Observation

The captured traffic contained established TCP connections. This indicates that TCP was actively being used for network communication during the capture.

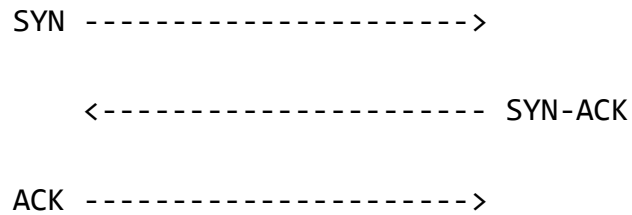
9. TCP 3-Way Handshake

TCP normally establishes a connection using a three-step handshake.

The process is:

Client

Server



Step 1 – SYN

The client sends a SYN packet to request a TCP connection.

Step 2 – SYN-ACK

The server responds with SYN-ACK.

This acknowledges the client's request and indicates that the server is ready to establish the connection.

Step 3 – ACK

The client sends an ACK packet.

The TCP connection is then established and application data can be exchanged.

9.1 Captured TCP Stream Analysis

A TCP stream was isolated using:

```
tcp.stream eq <stream_number>
```

The selected stream contained established TCP communication, including ACK and application-data packets.

However, the complete:

SYN → SYN-ACK → ACK

sequence was not available in the selected stream.

This is because the capture began after the TCP connection had already been established.

This is an important limitation when performing live packet captures: if a connection exists before the capture starts, its initial handshake packets may not be included.

Screenshot 5 – TCP Analysis

Insert: 05_tcp_handshake.png

Caption:

Figure 5: TCP stream analyzed using Wireshark showing established TCP communication.

Result

TCP communication was successfully identified and analyzed. The selected stream contained ACK and application-data packets, but a complete TCP handshake was not available in that particular stream.

10. Unencrypted Traffic Analysis

An attempt was made to identify unencrypted HTTP traffic using:

http

No plain HTTP packets were identified in the captured traffic.

Therefore, no real HTTP GET request containing readable application data was available for analysis.

Screenshot 6 – HTTP Analysis

Insert: 06_http_unencrypted.png

Caption:

Figure 6: Wireshark HTTP filter showing that no plain HTTP packets were identified.

Observation

The captured web traffic primarily consisted of encrypted TLS/HTTPS communication.

Therefore, the capture demonstrated the prevalence of encrypted web traffic rather than providing an example of readable HTTP application data.

11. Why HTTP is Dangerous

HTTP does not provide encryption for application data.

When HTTP traffic travels through an untrusted network, a network observer may potentially inspect the transmitted information.

Depending on the application, exposed information may include:

- URLs
- Web content
- Form data
- Cookies
- Session information
- Other application-layer data

This can create security risks such as:

- Eavesdropping
- Data interception
- Man-in-the-middle attacks
- Session compromise
- Data manipulation

Therefore, sensitive information should not be transmitted using plain HTTP.

12. How HTTPS Prevents Eavesdropping

HTTPS is HTTP protected by **TLS (Transport Layer Security)**.

HTTPS provides three important security properties.

Confidentiality

TLS encrypts application data, making it difficult for passive observers to read the content.

Integrity

TLS provides protection against unauthorized modification of data during transmission.

Authentication

TLS certificates help the client verify the identity of the server.

Therefore, HTTPS provides significantly stronger protection for web communication than plain HTTP.

13. HTTP vs HTTPS

Feature	HTTP	HTTPS
Encryption	No	Yes
Confidentiality	No protection	Protected
Integrity	No TLS protection	Protected by TLS
Server authentication	Not provided by HTTP	Supported through TLS certificates
Default Port	80	443
Recommended for sensitive data	No	Yes

14. Wireshark Filters Used

TCP

tcp

Used to display TCP packets.

DNS

`dns`

Used to display DNS queries and responses.

HTTP

`http`

Used to identify plain HTTP packets.

TCP Stream

`tcp.stream eq <stream_number>`

Used to isolate a specific TCP conversation.

SYN Packets

`tcp.flags.syn == 1 && tcp.flags.ack == 0`

Used to identify TCP SYN packets.

SYN-ACK Packets

`tcp.flags.syn == 1 && tcp.flags.ack == 1`

Used to identify SYN-ACK packets.

15. Key Observations

The following observations were made during the practical:

1. Wireshark was successfully installed and configured.
2. The Wi-Fi interface successfully captured live traffic.
3. DNS traffic was identified using the `dns` filter.
4. TCP traffic was identified using the `tcp` filter.
5. TLS traffic was observed in the captured communication.
6. No plain HTTP packets were identified.
7. The selected TCP stream contained established TCP traffic.

8. A complete TCP 3-way handshake was not available in the selected stream because the connection was already established when the capture was analyzed.
9. HTTPS/TLS traffic demonstrated the use of encrypted communication.

16. Security Considerations

Packet capture can potentially expose sensitive information.

Therefore, network monitoring should only be performed on:

- Personal networks
- Personal devices
- Authorized laboratory environments
- Networks where explicit permission has been obtained

Unauthorized packet capture on public, university, workplace, or other people's networks may violate privacy and security policies.

During this practical, the analysis was performed for educational and authorized purposes.

17. Glossary

Packet

A small unit of data transmitted across a computer network.

Protocol

A defined set of rules that controls communication between network devices.

Port

A logical endpoint used to identify a network service or application.

Payload

The actual data carried inside a packet or transport segment.

Handshake

An initial exchange of messages used to establish or coordinate communication between network endpoints.

18. Capture File

The captured network traffic was exported in PCAP format.

File name:

wireshark_capture.pcap

The capture file can be reopened in Wireshark for further packet-level analysis.

19. Ethical Considerations

This practical was performed only for educational and authorized network-analysis purposes.

Network traffic should only be captured when the analyst owns the network/device or has explicit authorization.

Sensitive information such as:

- Passwords
- Banking information
- Private messages
- Authentication tokens
- Personal communications

should not be intentionally captured or disclosed.

20. References

1. Wireshark Official Website

<https://www.wireshark.org/>

2. Wireshark User Guide

https://www.wireshark.org/docs/wsug_html/

3. Wireshark Wiki

<https://wiki.wireshark.org/>

4. RFC 9293 – Transmission Control Protocol

<https://www.rfc-editor.org/rfc/rfc9293>

5. RFC 9110 – HTTP Semantics

<https://www.rfc-editor.org/rfc/rfc9110>

6. RFC 8446 – TLS 1.3

<https://www.rfc-editor.org/rfc/rfc8446>

Final Submission Checklist

- Wireshark installed
- Live traffic captured
- HTTP filter tested
- DNS filter tested
- TCP traffic analyzed
- TCP handshake concept documented
- HTTPS security analyzed
- Ethical considerations documented
- Screenshots prepared
- wireshark_capture.pcap exported
- README prepared
- Final report prepared